Module 6

Deep Reinforcement Learning

Selina Ochukut

Module Overview

This module introduces **reinforcement learning (RL)**: the branch of machine learning concerned with an *agent* learning to make sequential decisions by interacting with an *environment* to maximize cumulative reward, without being told the correct action at any given moment. We begin with the classical mathematical framework of RL (Markov Decision Processes, value functions, the Bellman equations), then examine why deep neural networks are needed to scale RL to large and continuous problems. You will explore **Deep Q-Networks (DQN)**, the algorithm that first combined deep learning with RL at scale, followed by **policy-based and actor-critic methods**, which optimize a policy directly rather than a value function. You will then survey the most influential **advanced algorithms** used in modern practice (TRPO, PPO, DDPG, SAC), and close with the **practical applications and open challenges** of deploying deep RL in the real world, including its central role in aligning today's large language models.

Expected Learning Outcomes

- **Explain** the fundamental concepts of reinforcement learning.
- **Apply** reinforcement learning algorithms to solve sequential decision-making problems in simulated environments.
- **Analyze** the theoretical foundations and performance trade-offs of modern deep reinforcement learning algorithms.
- **Evaluate** the suitability of different deep reinforcement learning approaches for real-world applications.

1 Introduction to Reinforcement Learning

1.1 The Reinforcement Learning Problem

Reinforcement learning studies how an **agent** should choose **actions** within an **environment**, over a sequence of time steps, in order to maximize the total **reward** it receives. Unlike supervised learning, the agent is never told the "correct" action directly – it only receives a scalar reward signal, and must discover good behavior through trial and error.

Definition

At each discrete time step t, the agent:

1. Observes the environment's current **state** $s_t \in \mathcal{S}$.
2. Selects an **action** $a_t \in \mathcal{A}$ according to its **policy** π.
3. Receives a scalar **reward** r_t and the environment transitions to a new state s_{t+1}, according to the environment's (possibly unknown) dynamics.

This repeats, generating a **trajectory** $s_0, a_0, r_0, s_1, a_1, r_1, \ldots$. The agent's goal is to learn a policy that maximizes the expected cumulative reward over time.

1.2 Markov Decision Processes

Video Visit the URL below to view a video:

https://www.youtube.com/embed/sWKSOY9DCiI

Definition

The standard mathematical formalization of the RL problem is a **Markov Decision Process (MDP)**, defined by a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- $\mathcal{S}$: the set of possible states.
- $\mathcal{A}$: the set of possible actions.

- $P(s' \mid s, a)$: the **transition function**, giving the probability of moving to state s' after taking action a in state s.
- $R(s, a, s')$ (or simply r): the **reward function**.
- $\gamma \in [0, 1)$: the **discount factor**, controlling how much future rewards are worth relative to immediate rewards.

Key Idea

The defining structural assumption of an MDP is the **Markov property**: the probability of the next state and reward depends only on the *current* state and action, not on the full history that led there, i.e., $P(s_{t+1} \mid s_t, a_t) = P(s_{t+1} \mid s_0, a_0, \ldots, s_t, a_t)$. This assumption is what makes the problem tractable: the current state is assumed to be a *sufficient statistic* for all decision-relevant information about the past, so the agent never needs to remember its full history, only its current state.

1.3 Returns, Policies, and Value Functions

Definition

The **return** G_t is the total discounted reward accumulated from time t onward:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

A **policy** $\pi(a \mid s)$ specifies the probability of taking action a in state s (a **deterministic policy** instead directly maps $s \mapsto a$). The **state-value function** and **action-value function** (or Q-function) under policy π are:

$$V^\pi(s) = \mathbb{E}_\pi\big[G_t \mid s_t = s\big], \qquad Q^\pi(s, a) = \mathbb{E}_\pi\big[G_t \mid s_t = s, a_t = a\big]$$

Key Idea

The discount factor γ serves two purposes: it keeps the (potentially infinite) sum G_t mathematically finite whenever rewards are bounded, and it encodes a preference for sooner rewards over later ones (a smaller

γ makes the agent more "myopic," a γ close to 1 makes it consider long-term consequences more heavily). $V^\pi(s)$ tells us how good it is to *be* in state s under policy π; $Q^\pi(s,a)$ tells us how good it is to *take action* a *in state* s, and then follow π thereafter – this distinction matters because Q^π directly supports action selection ($\arg\max_a Q^\pi(s,a)$), while V^π alone does not, without also knowing the environment's transition dynamics.

1.4 The Bellman Equations

Video Visit the URL below to view a video:

https://www.youtube.com/embed/9JZID-h6ZJ0

Definition

The value functions satisfy a recursive relationship known as the **Bellman expectation equation**, which decomposes the value of a state into the immediate reward plus the discounted value of the next state:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s,a)\big[R(s,a,s') + \gamma V^\pi(s')\big]$$

The **optimal value function** $V^*(s) = \max_\pi V^\pi(s)$ and optimal action-value function $Q^*(s,a) = \max_\pi Q^\pi(s,a)$ satisfy the **Bellman optimality equation**:

$$Q^*(s,a) = \sum_{s'} P(s' \mid s,a)\Big[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\Big]$$

Key Idea

The Bellman optimality equation is the theoretical foundation of nearly every value-based RL algorithm, including DQN (Section 3): it expresses the optimal Q-value of a state-action pair purely in terms of the immediate reward and the optimal Q-value of the *next* state, *without* needing to know the value of every possible future trajectory directly. This recursive, one-step-lookahead structure is what makes value func-

tions learnable via **bootstrapping** – updating an estimate of $Q(s, a)$ using another (current) estimate of $Q(s', a')$, rather than requiring a full trajectory to complete before any learning can happen.

1.5 Exploration vs. Exploitation

Key Idea

An RL agent faces a fundamental dilemma at every decision: should it **exploit** its current knowledge by choosing the action it currently believes is best, or should it **explore** an action it knows less about, in case that action is actually better? Pure exploitation risks getting permanently stuck with a suboptimal policy based on incomplete information; pure exploration never capitalizes on what has been learned. A common simple strategy is ϵ**-greedy** exploration: choose the current best-known action with probability $1 - \epsilon$, and a uniformly random action with probability ϵ, often decaying ϵ over training as the agent's estimates become more reliable.

1.6 Model-Based vs. Model-Free, On-Policy vs. Off-Policy

- **Model-based RL:** The agent learns (or is given) the transition function P and reward function R, and can use them to plan (e.g., simulate hypothetical trajectories) without needing to interact with the real environment for every update.
- **Model-free RL:** The agent learns a value function and/or policy directly from experience, without ever explicitly modeling P or R. Most of the algorithms in this module (DQN, policy gradients, actor-critic) are model-free.
- **On-policy learning:** The agent updates its value function/policy using data generated by the *current* policy being evaluated or improved (e.g., SARSA, REINFORCE, PPO).
- **Off-policy learning:** The agent can learn about one policy (the **target policy**, often the optimal policy) using data generated by a *different* policy (the **behavior policy**), which is what allows techniques like experience replay (Section 3) to reuse old data (e.g., Q-learning, DQN, DDPG).

2 Introduction to Deep Reinforcement Learning

2.1 Why Tabular RL Does Not Scale

Classical RL algorithms (value iteration, policy iteration, tabular Q-learning) represent $V(s)$ or $Q(s, a)$ as an explicit table, with one entry per state (or state-action pair). This works well for small, discrete problems, but breaks down as problems grow realistic.

Key Idea

Real-world problems often have enormous or continuous state spaces: an Atari game screen has more possible pixel configurations than atoms in the observable universe, and a robot's joint angles and velocities are continuous real numbers. A tabular representation cannot store a value for every possible state, and even if it theoretically could, it would need to visit each state individually to learn its value, which is hopelessly sample-inefficient. **Function approximation** – representing V, Q, or the policy π using a parametric function (most powerfully, a neural network) rather than a table – solves both problems: the function can generalize its predictions to states it has never exactly visited before, based on similarity to states it has seen.

2.2 Deep RL: Function Approximation Meets RL

Definition

Deep reinforcement learning refers to RL algorithms in which the value function, policy, or both are represented by **deep neural networks**: $Q(s, a; \theta)$, $V(s; \theta)$, or $\pi(a \mid s; \theta)$, where θ denotes the network's learnable weights. The network takes the (possibly high-dimensional, raw) state as input – e.g., raw pixels – and outputs the corresponding value(s) or action probabilities, learned end-to-end via gradient-based optimization.

2.3 New Challenges Introduced by Function Approximation

Combining deep learning with RL introduces instabilities absent from the tabular setting, most famously summarized as the **deadly triad**.

Key Idea

The **deadly triad** refers to the observation that combining (1) **function approximation**, (2) **bootstrapping** (updating a value estimate using another current value estimate, as in the Bellman equation), and (3) **off-policy learning**, can cause training to diverge – each of the three elements is individually useful and often necessary, but their combination lacks the convergence guarantees that hold when only one or two are present. Since most practical deep RL systems require all three (neural networks for scale, bootstrapping for sample efficiency, off-policy learning for data reuse), a large fraction of deep RL algorithm design is specifically about techniques to *stabilize* training despite this theoretical risk of divergence.

Additional practical challenges specific to the deep RL setting include:

- **Correlated, non-i.i.d. data:** Unlike supervised learning, where training examples are typically assumed independent and identically distributed, consecutive states visited by an RL agent are highly correlated (each state is usually similar to the previous one), which violates the assumptions most gradient-based optimizers are designed around and can bias or destabilize learning.
- **Non-stationary targets:** In value-based methods, the "target" value used to compute the training loss itself depends on the network's own (constantly changing) current parameters, creating a moving target that a purely supervised-learning mindset does not need to handle.
- **Sparse and delayed rewards:** A reward signal may only appear many steps after the action(s) responsible for it, creating a difficult **credit assignment** problem: which of the many preceding actions actually caused the eventual reward?

2.4 The General Deep RL Training Loop

Despite the diversity of specific algorithms, most deep RL methods share a common high-level training loop:

1. **Interact:** The agent selects actions (using its current policy, typically with some exploration) and executes them in the environment, observing rewards and next states.

2. **Store:** Experience (transitions (s, a, r, s'), or full trajectories) is collected, sometimes stored in a buffer for later reuse (Section 3).

3. **Update:** One or more neural networks (value function, policy, or both) are updated using gradient descent on a loss function derived from the Bellman equation, the policy gradient theorem, or a related objective.

4. **Repeat,** with the (now updated) policy generating new experience, and the cycle continuing until performance converges.

3 Deep Q-Networks (DQN)

Deep Q-Networks (Mnih et al., 2015) were the first algorithm to demonstrate that deep learning and RL could be combined stably at scale, famously learning to play dozens of Atari 2600 games directly from raw pixels, often exceeding human performance, using a single architecture and hyperparameter set.

3.1 From Q-Learning to Deep Q-Learning

Classical (tabular) **Q-learning** updates a table entry via:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big]$$

where the bracketed term is the **TD (temporal-difference) error**. DQN replaces the table with a neural network $Q(s,a;\theta)$, trained to minimize the squared TD error via gradient descent:

$$L(\theta) = \mathbb{E}_{(s,a,r,s')}\Big[\big(\underbrace{r + \gamma \max_{a'} Q(s',a';\theta)}_{\text{target}} - Q(s,a;\theta)\big)^2\Big]$$

Key Idea

Naively applying this loss with a single neural network, updated online from consecutive experience, is highly unstable in practice – exactly the deadly-triad instability discussed in Section 2. DQN's key contribution was two specific architectural tricks that make this combination work reliably.

3.2 Trick 1: Experience Replay

Definition

An **experience replay buffer** stores past transitions (s,a,r,s') in a large, fixed-size memory. Instead of updating the network using only the most recent transition, DQN samples a random **minibatch** of past transitions from this buffer at every update step.

Key Idea

Experience replay directly addresses the correlated-data problem: by sampling a random minibatch of transitions from many different points in the agent's history, consecutive gradient updates are computed on far more diverse, decorrelated data than the highly similar consecutive states the agent would otherwise see. It also improves sample efficiency, since each transition can be reused in multiple updates rather than being discarded immediately after use – something only possible because Q-learning is **off-policy** (Section 1.6): the transitions in the buffer may have been generated by an older version of the policy, but they remain valid data for learning about the current target policy.

3.3 Trick 2: Target Network

Definition

DQN maintains *two* copies of the Q-network: the **online network** $Q(s, a; \theta)$, updated at every step, and a **target network** $Q(s, a; \theta^-)$, whose weights θ^- are only periodically copied from θ (e.g., every few thousand steps) or updated via a slow exponential moving average. The training target uses the *target* network:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

Key Idea

Without a separate target network, the bootstrapped target $r + \gamma \max_{a'} Q(s', a'; \theta)$ changes at *every single gradient step*, since it uses the same rapidly-changing parameters θ that are simultaneously being updated – the network is effectively chasing a target that moves every time it takes a step toward it, which can cause oscillation or divergence. Freezing the target network's parameters for many steps at a time gives the online network a stable, fixed objective to converge toward for a while, dramatically improving training stability.

3.4 The DQN Algorithm

1. Initialize online network θ and target network $\theta^- \leftarrow \theta$; initialize an empty replay buffer.

2. For each environment step: select action a_t via ϵ-greedy applied to $Q(s_t, \cdot; \theta)$; execute it; store (s_t, a_t, r_t, s_{t+1}) in the replay buffer.

3. Sample a random minibatch of transitions from the buffer; compute the target $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$ (or just r if s' is terminal).

4. Update θ by gradient descent on $(y - Q(s, a; \theta))^2$.

5. Periodically synchronize $\theta^- \leftarrow \theta$.

3.5 Improvements to DQN

3.5.1 Double DQN

Key Idea

Standard Q-learning's $\max_{a'} Q(s', a'; \theta^-)$ operation uses the *same* network both to *select* which action looks best and to *evaluate* that action's value, which systematically **overestimates** Q-values whenever the network's value estimates are noisy (since taking a max over noisy estimates is biased upward). **Double DQN** (van Hasselt et al., 2016) decouples selection and evaluation: it uses the *online* network to select the best next action, but the *target* network to evaluate that action's value:

$$y = r + \gamma\, Q\big(s', \arg\max_{a'} Q(s', a'; \theta);\ \theta^-\big)$$

This simple change substantially reduces overestimation bias and improves final performance in practice.

3.5.2 Dueling DQN

Dueling DQN (Wang et al., 2016) changes the network's *architecture* rather than its training target: instead of directly outputting $Q(s, a)$ for each action, the network computes two separate streams – a scalar state-value $V(s)$ and a per-action **advantage** $A(s, a)$ – which are then combined:

$$Q(s, a) = V(s) + \Big(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')\Big)$$

Key Idea

This decomposition is useful because, in many states, the specific action taken barely matters for the eventual outcome (e.g., no obstacle

nearby), and it is more sample-efficient to learn the shared state-value $V(s)$ once than to separately learn very similar Q-values for every action in that state. The mean-subtraction in the combination formula is a normalization needed to keep $V(s)$ and $A(s, a)$ individually identifiable (otherwise, a constant could be added to V and subtracted from every $A(s, a)$ with no change to Q, making the decomposition ambiguous).

3.5.3 Prioritized Experience Replay

Rather than sampling uniformly from the replay buffer, **Prioritized Experience Replay** (Schaul et al., 2016) samples transitions with probability proportional to the magnitude of their (most recent) TD error, so that the network spends more training time on transitions it currently predicts poorly (and is therefore likely to learn the most from), rather than wasting equal effort on transitions it already handles well.

3.5.4 Rainbow

Rainbow (Hessel et al., 2018) combines Double DQN, Dueling DQN, Prioritized Experience Replay, and several other independently-developed improvements into a single agent, and demonstrated that these enhancements are largely complementary, together substantially outperforming any single improvement in isolation.

4 Policy-Based and Actor-Critic Methods

DQN and its variants are **value-based**: they learn a Q-function, and the policy is only implicitly defined by acting greedily with respect to it. **Policy-based methods** instead directly parameterize and optimize a policy $\pi(a \mid s; \theta)$, without necessarily learning a value function at all.

4.1 Why Optimize the Policy Directly?

Key Idea

Value-based methods like DQN require computing $\arg\max_a Q(s, a)$, which is straightforward when $\mathcal{A}$ is small and discrete, but becomes intractable when actions are **continuous** (e.g., a robot's continuous joint torques) – there is no simple way to maximize over an infinite, continuous action set at every step. Policy-based methods sidestep this entirely by having the network directly output an action (or a distribution over actions), naturally supporting continuous action spaces. Policy-based methods can also learn genuinely **stochastic** policies (useful when the optimal behavior is inherently random, e.g., in adversarial or partially observable settings), whereas a value-based method's greedy policy is deterministic by construction (aside from exploration noise added on top).

4.2 The Policy Gradient Theorem

Definition

Let $J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$ be the expected return under policy π_θ. The **policy gradient theorem** states that the gradient of this objective with respect to the policy parameters θ can be written as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t\Big]$$

Key Idea

This remarkable result says that we can improve the policy using only the *gradient of the log-probability* of the actions actually taken, weighted by how good the resulting return turned out to be – *without* needing to differentiate through the (possibly unknown, non-

differentiable) environment dynamics at all. Intuitively: actions that led to high returns should be made *more* likely in the future (increase $\log \pi_\theta(a_t \mid s_t)$ in that direction); actions that led to low or negative returns should be made less likely. This is the theoretical foundation of every policy-based and actor-critic method in this section.

4.3 REINFORCE

Definition

REINFORCE (Williams, 1992) is the most direct application of the policy gradient theorem: collect a full trajectory (episode) using the current policy, compute the actual return G_t obtained from each time step onward, and update the policy parameters via gradient ascent:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t$$

Key Idea

REINFORCE is simple and unbiased, but suffers from very **high variance**: the return G_t depends on an entire (often long, stochastic) trajectory, so two runs starting from similar states can yield wildly different G_t values purely by chance, making the gradient estimate noisy and slow to converge. A standard **variance-reduction** technique is to subtract a **baseline** $b(s_t)$ (commonly an estimate of $V(s_t)$) from the return, using $(G_t - b(s_t))$ in place of G_t: this can be shown not to introduce any bias into the gradient estimate (in expectation, the baseline term contributes zero), while substantially reducing its variance, since it centers the learning signal around zero for an "average" outcome and only reinforces / penalizes performance relative to that baseline.

4.4 Actor-Critic Methods

Definition

An **actor-critic** method maintains two components, typically both parameterized by neural networks:

- The **actor**: the policy $\pi_\theta(a \mid s)$, updated using the policy gradient.

- The **critic**: a value function $V_\phi(s)$ (or $Q_\phi(s,a)$), trained via TD learning/bootstrapping (much like DQN's value learning), used to provide a *lower-variance* estimate of how good an action was, in place of the full Monte Carlo return G_t.

Key Idea

Actor-critic methods combine the best of both method families studied so far: like policy-based methods, they directly optimize a policy and naturally handle continuous/stochastic actions; like value-based methods, they use bootstrapping (a learned value estimate) rather than waiting for a full episode's return, which substantially reduces variance and allows learning from incomplete trajectories. The critic does not choose actions itself – its sole job is to evaluate the actor's choices and provide a better training signal than the raw, noisy Monte Carlo return.

4.5 The Advantage Function

Definition

The **advantage function** $A(s,a) = Q(s,a) - V(s)$ measures how much better (or worse) action a is compared to the *average* action in state s (under the current policy). Using $A(s,a)$ in place of G_t or $Q(s,a)$ in the policy gradient gives:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A(s_t, a_t)\Big]$$

In practice, $A(s_t, a_t)$ is commonly estimated using the one-step TD error, $\hat{A}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$, which requires learning only V_ϕ, not a separate Q_ϕ.

Key Idea

The advantage function is exactly the baseline-subtraction idea from Section 4.3, formalized: subtracting $V(s)$ from $Q(s,a)$ centers the learning signal on zero for an average action, so the policy gradient only pushes probability mass toward genuinely *above-average* actions and away from below-average ones, which is both unbiased and sub-

stantially lower-variance than using the raw return.

4.6 A2C and A3C

Advantage Actor-Critic (A2C) is a practical, synchronous implementation of the actor-critic framework using the advantage function above. **Asynchronous Advantage Actor-Critic (A3C)** (Mnih et al., 2016) runs many parallel actor-learner instances (each with its own copy of the environment) simultaneously on different CPU threads, each independently computing gradients and asynchronously applying them to a shared set of parameters.

Key Idea

A3C's parallel workers naturally generate diverse, decorrelated experience simply by exploring independently from different points in the state space at the same time – providing an alternative solution to the correlated-data problem (Section 2), without needing an experience replay buffer at all. This also makes A3C (and its variants) naturally on-policy and compatible with algorithms that cannot easily reuse old off-policy data.

5 Advanced Deep Reinforcement Learning Methods

Building on the actor-critic foundation, this section surveys the algorithms most widely used in current practice, each addressing a specific limitation of the basic policy gradient / actor-critic framework.

5.1 Trust Region Policy Optimization (TRPO)

Key Idea

A plain policy gradient step can, if the learning rate is even slightly too large, push the policy parameters far enough to catastrophically worsen performance – and because the *next* batch of training data is collected using this now-worse policy, a single bad update can be very difficult to recover from (unlike supervised learning, where a bad gradient step just means slightly worse test accuracy this epoch, not corrupted future training data). **TRPO** (Schulman et al., 2015) addresses this by constraining each policy update to stay within a **trust region**: a bound on how much the new policy $\pi_{\theta_{new}}$ is allowed to differ from the old policy $\pi_{\theta_{old}}$, measured via the KL divergence $D_{KL}(\pi_{\theta_{old}} \| \pi_{\theta_{new}}) \leq \delta$, solved as a constrained optimization problem at every update.

5.2 Proximal Policy Optimization (PPO)

Key Idea

TRPO's constrained optimization is theoretically well-motivated but computationally involved (it requires approximating second-order curvature information). **PPO** (Schulman et al., 2017) achieves a similar effect – preventing overly large, destructive policy updates – using a much simpler, first-order-only **clipped surrogate objective**, making it dramatically easier to implement while retaining most of TRPO's stability benefits. This simplicity is a major reason PPO became, and remains, one of the most widely used RL algorithms in practice, including as a core component of RLHF pipelines for large language models (Section 6).

Definition

Let $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ be the probability ratio between the new and old policy for the action actually taken. PPO's clipped objective is:

$$L^{CLIP}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big]$$

where $\hat{A}_t$ is an estimated advantage and ϵ (e.g., 0.2) is a small hyperparameter.

Key Idea

The clipping term removes the incentive for the optimizer to push $r_t(\theta)$ far outside the range $[1-\epsilon, 1+\epsilon]$: if an update would make an already-good action ($\hat{A}_t > 0$) even *more* likely beyond this range, the clipped term caps the objective's benefit from doing so, so gradient ascent has no further incentive to push the ratio higher; similarly for a bad action, the ratio is prevented from being pushed too low in a single step. This keeps each policy update "proximal" (close) to the previous policy, without any explicit KL-divergence constraint or second-order computation.

5.3 Deep Deterministic Policy Gradient (DDPG)

DDPG (Lillicrap et al., 2015) extends the actor-critic framework to continuous action spaces using a *deterministic* policy $\mu_\theta(s)$ (rather than a distribution), combined with several of DQN's stabilization techniques.

Definition

DDPG maintains an actor $\mu_\theta(s)$ (outputting a specific continuous action) and a critic $Q_\phi(s, a)$ (estimating the action-value of any continuous action, not just discrete ones). It is trained off-policy using an experience replay buffer and target networks for both the actor and critic (exactly as in DQN), with the critic trained to minimize the Bellman error and the actor trained to maximize $Q_\phi(s, \mu_\theta(s))$ directly via the chain rule through the critic.

Key Idea

DDPG illustrates that the stabilization tricks introduced for DQN (Section 3) – experience replay and target networks – are not specific to discrete-action, value-based methods; they generalize naturally to continuous-action actor-critic methods as well, since the same deadly-triad instabilities (function approximation + bootstrapping + off-policy learning) are present in both settings.

5.4 Soft Actor-Critic (SAC)

SAC (Haarnoja et al., 2018) is a further refinement for continuous control, built around the idea of **maximum entropy RL**.

Definition

Maximum entropy RL augments the standard RL objective with an entropy bonus, encouraging the policy to act as randomly as possible while still achieving high reward:

$$J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\sum_t r_t + \alpha\,\mathcal{H}\big(\pi_\theta(\cdot \mid s_t)\big)\Big]$$

where $\mathcal{H}(\pi_\theta(\cdot \mid s_t))$ is the entropy of the policy's action distribution at state s_t, and α controls the trade-off between reward-seeking and randomness.

Key Idea

Explicitly rewarding entropy serves two purposes: it encourages more thorough **exploration** throughout training (since a higher-entropy policy naturally tries a wider variety of actions), and it produces a more **robust** final policy that does not collapse onto a single, narrow behavior as soon as it finds *any* reward-earning strategy, remaining better prepared to handle small variations in the environment. SAC combines this maximum-entropy objective with an off-policy actor-critic architecture (similar in spirit to DDPG), and has become one of the most reliable and widely used algorithms for continuous-control tasks.

5.5 Model-Based Deep RL

All algorithms discussed so far are model-free. **Model-based** deep RL methods additionally learn an explicit model of the environment's dynamics, $\hat{P}(s' \mid s, a)$ (and often $\hat{R}(s, a)$), typically using a neural network trained via supervised learning on observed transitions.

Key Idea

Once a reasonably accurate dynamics model is learned, it can be used to generate additional simulated experience (without needing to interact with the real environment further) or to explicitly plan ahead by simulating the consequences of candidate action sequences before committing to one – both of which can substantially improve **sample efficiency**, which is one of model-free RL's biggest practical weaknesses (Section 6). The trade-off is that model-based methods are only as good as their learned model: errors in the dynamics model can compound over multi-step simulated rollouts, potentially misleading the planning or policy-learning process if not carefully managed.

6 Practical Applications and Challenges of Deep RL

6.1 Notable Applications

- **Game playing:** DQN on Atari 2600 games (Section 3); **AlphaGo** and **AlphaZero** (Silver et al., 2016, 2017), which combined deep RL with Monte Carlo Tree Search to defeat top human players at Go, and later mastered Go, chess, and shogi entirely through self-play with no human game data; and **OpenAI Five** / **AlphaStar**, which reached professional-level play in the complex, long-horizon, partially-observable games Dota 2 and StarCraft II.
- **Robotics and continuous control:** DDPG, SAC, and PPO are widely used to learn robotic locomotion and manipulation policies, both in simulation and (with additional care, see Section 6.2) on physical hardware.
- **Recommendation systems:** RL formulations that treat a sequence of recommendations as a trajectory, aiming to maximize long-term user engagement rather than a single immediate click.
- **Resource management and control:** Data center cooling optimization, chip placement, and traffic signal control have all been formulated as RL problems.
- **RLHF for large language models: Reinforcement Learning from Human Feedback** trains a reward model from human preference comparisons between candidate model outputs, then fine-tunes a language model (frequently using PPO, given the discussion in Section 5) to maximize this learned reward – one of the most impactful recent applications of deep RL, directly underlying the alignment process of modern conversational AI systems.

6.2 Core Challenges

- **Sample inefficiency:** Many deep RL algorithms require millions to billions of environment interactions to learn effective policies, which is often practical in fast, cheap simulators (games) but prohibitively slow or expensive in the physical world (robotics), motivating model-based methods and other sample-efficiency techniques.
- **Sparse and delayed rewards / credit assignment:** When useful reward signal is rare (e.g., only upon completing an entire complex task),

the agent receives very little training signal for most of its experience, and must somehow determine which of many earlier actions were actually responsible for a later reward.

- **Exploration in large or sparse-reward environments:** Simple exploration strategies like ϵ-greedy are often inadequate when a task requires a long, specific sequence of actions before any reward is ever observed (e.g., simple random exploration is exceedingly unlikely to stumble upon the exact sequence needed to complete a complex game level), motivating more sophisticated exploration techniques (e.g., intrinsic curiosity rewards, count-based exploration bonuses).
- **Reward hacking / specification gaming:** An agent optimizing a reward function exactly as specified may discover unintended strategies that technically maximize the reward while failing to achieve the designer's actual intent (e.g., a boat-racing agent that loops through reward-granting checkpoints indefinitely, rather than finishing the race), highlighting the difficulty of precisely specifying complex real-world objectives.
- **Sim-to-real gap:** Policies trained in simulation frequently perform substantially worse when deployed on real physical hardware, since simulators can never perfectly capture every detail of real-world physics, sensor noise, and actuator behavior.
- **Reproducibility and sensitivity to hyperparameters:** Deep RL training is notoriously sensitive to random seeds, hyperparameters, and even minor implementation details, and published results can be difficult to reproduce, complicating both research progress and practical deployment decisions.
- **Safety and reliability:** Since RL agents actively explore and can take unexpected actions, deploying them in safety-critical settings (autonomous vehicles, healthcare, physical robots) requires additional guarantees or constraints well beyond simply maximizing expected reward.

Key Idea

A common thread running through nearly all of these challenges is that deep RL agents optimize *exactly* what they are told to optimize (the specified reward function), not necessarily what their designer actually intended – and the gap between these two things (specification gaming, reward hacking) becomes more consequential as deep RL systems are deployed with greater autonomy and in higher-stakes settings, in-

cluding the RLHF pipelines that shape the behavior of widely-used AI systems.

REFERENCES

- Prince, Simon JD. Understanding deep learning. MIT press, 2023.
- Goodfellow, I., Bengio, Y., Courville, A., & Bengio, Y. (2016). Deep learning (Vol. 1, No. 2, pp. 1-800). Cambridge: MIT press. https://www.deeplearningbook.org/
- Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Vol. 1. Cham, Switzerland: Springer, 2024.
- Torralba, A., Isola, P., & Freeman, W. T. (2024). Foundations of computer vision. MIT Press.